

Name : Vishva M Bhimani RollNo: 315 24010101026

Practical 1: Student Marks Analyzer

Problem Definition

Store student data as a list of tuples. Remove duplicate marks, calculate average marks and store result in a dictionary.

Sample Input

```
students = [  
    (1, "Amit", [78, 85, 90, 85]),  
    (2, "Neha", [88, 92, 79, 88]),  
    (3, "Ravi", [70, 75, 80, 75])  
]
```

Sample Output

```
{1: ('Amit', 84.33), 2: ('Neha', 86.33), 3: ('Ravi', 75.0)}
```

```
In [36]: students = [  
    (1, "Amit", [78, 85, 90, 85]),  
    (2, "Neha", [88, 92, 79, 88]),  
    (3, "Ravi", [70, 75, 80, 75])  
]  
  
res = {}  
  
for i in students:  
    roll, name, marks = i  
    newM = list(set(marks))  
    avrag = sum(newM)/len(newM)  
    new = round(avrag, 2)  
    res[roll] = (name, new)  
print(res)
```

```
{1: ('Amit', 84.33), 2: ('Neha', 86.33), 3: ('Ravi', 75.0)}
```

Practical 2: Inventory Management

Problem Definition

Remove out-of-stock items and calculate total inventory value.

Sample Input

```
inventory = {
    'pen': (10, 50),
    'book': (100, 0),
    'pencil': (5, 100)
}
```

Sample Output

```
{'pen': (10, 50), 'pencil': (5, 100)}
Total Value: 1000
```

```
In [49]: inventory = {
    'pen': (10, 50),
    'book': (100, 0),
    'pencil': (5, 100)
}

d1 = {item: details for item, details in inventory.items() if details[1] > 0}
print(d1)
total_value = sum(q * price for q, price in d1.values())
print(f"Total Value: {total_value}")
```

```
{'pen': (10, 50), 'pencil': (5, 100)}
Total Value: 1000
```

Practical 3: Email Domain Extractor

Problem Definition

Extract email domains and count occurrences.

Sample Input

```
emails = ['a@gmail.com', 'b@yahoo.com', 'c@gmail.com']
```

Sample Output

```
{'gmail.com': 2, 'yahoo.com': 1}
```

```
In [58]: emails = ['a@gmail.com', 'b@yahoo.com', 'c@gmail.com']
domain_counts = {}
```

```

for i in emails:
    domain = i.split('@')[1]
    if domain in domain_counts:
        domain_counts[domain] += 1
    else:
        domain_counts[domain] = 1

print(domain_counts)

```

```
{'gmail.com': 2, 'yahoo.com': 1}
```

Practical 4: Shopping Cart Billing

Problem Definition

Calculate item-wise cost and total bill.

Sample Input

```

cart = [('apple', 2), ('banana', 3)]
prices = {'apple': 50, 'banana': 10}

```

Sample Output

```

{'apple': 100, 'banana': 30}
Total Bill: 130

```

```

In [68]: cart = [('apple', 2), ('banana', 3)]
prices = {'apple': 50, 'banana': 10}
item_costs = {}
total_bill = 0
count=0
cost=0
pr=list(prices.values())
for item, quantity in cart:
    cost = pr[count] * quantity
    item_costs[item] = cost
    total_bill += cost
    count+=1
print(item_costs)
print("Total Bill:", total_bill)

```

```

{'apple': 100, 'banana': 30}
Total Bill: 130

```

Practical 5: Character Position Index

Problem Definition

Store positions of each character in a string.

Sample Input

```
s = "banana"
```

Sample Output

```
{'b': [0], 'a': [1, 3, 5], 'n': [2, 4]}
```

```
In [73]: s = "banana"
result = {}

for i, char in enumerate(s):
    if char not in result:
        result[char] = []
    result[char].append(i)

print(result)
```

```
{'b': [0], 'a': [1, 3, 5], 'n': [2, 4]}
```

Practical 6: Survey Data Aggregation

Problem Definition

Count responses city-wise and choice-wise.

Sample Input

```
responses = [
    ('Amit', 'Rajkot', 'Yes'),
    ('Neha', 'Rajkot', 'No'),
    ('Ravi', 'Ahmedabad', 'Yes')
]
```

Sample Output

```
{'Rajkot': {'Yes': 1, 'No': 1}, 'Ahmedabad': {'Yes': 1}}
```

```
In [75]: responses = [
    ('Amit', 'Rajkot', 'Yes'),
    ('Neha', 'Rajkot', 'No'),
```

```
    ('Ravi', 'Ahmedabad', 'Yes')
]

output = {}
for name, city, response in responses:

    if city not in output:
        output[city] = {}
    output[city][response] = output[city].get(response, 0) + 1

print(output)
```

```
{'Rajkot': {'Yes': 1, 'No': 1}, 'Ahmedabad': {'Yes': 1}}
```

In []: